

WindBorne Firmware Engineer Questions

Syamdilya Nimmagadda

1. A cool thing I built/coded up

One project I am especially proud of is my **SPI Slave with Flow Control** project. I implemented an SPI slave with protocol-level flow control using dual-clock RX/TX FIFOs, watermark-based backpressure, a BUSY-token path for not-ready reads, and status-first polling. What made it cool was that it was not just a basic SPI block; I had to think about real hardware behavior such as clock-domain crossing, FIFO underflow/overflow, timing, backpressure, and how software or a master device would interact with the hardware safely.

I also built a UVM-style testbench that randomly varied SPI modes and frame lengths, used a self-checking scoreboard, added SystemVerilog Assertions for CS/SCK timing, and wrote Python scripts to plot throughput and FIFO usage. This project is something I would show because it connects RTL design, verification, protocol behavior, timing, and debugging in one system.

2. My setup

My regular development setup is **VS Code**, Linux terminal, Git, and Python scripting. I like VS Code because it is lightweight, easy to customize, and works well across different types of work: Python scripts, Verilog/SystemVerilog files, documentation, Markdown, and Git workflows.

For hardware design and verification, I have used tools such as **Synopsys VCS**, **Xilinx Vivado**, **Quartus Prime**, **QuestaSim**, **SimVision**, **JasperGold**, **MATLAB**, and **Jupyter Notebook**. My setup has changed over time from “just write the code and simulate it” to a more structured workflow where I care about logs, repeatable runs, regression results, coverage reports, and documentation. I also like using small Python scripts to automate repetitive checks, compare register maps, parse logs, and summarize test results because it makes hardware debugging much faster and less manual.

3. PCB randomly resets once every few hours — how I would debug it

If a PCB resets randomly once every few hours, I would first avoid guessing and try to separate the problem into power, reset, firmware/software, hardware interface, thermal, and timing causes.

My first couple of tests would be:

1. **Check all power rails with an oscilloscope and DMM.** I would look for voltage dips, ripple, brownout conditions, regulator instability, or sudden load-related transients.
2. **Probe the reset line.** I would check if reset is being externally asserted or if the processor/FPGA is resetting internally.

3. **Check watchdog and boot reason.** If the system has a watchdog or reset-cause register, I would log whether the reset came from watchdog timeout, brownout, software reset, external reset, or unknown cause.
4. **Add heartbeat and state logging.** I would log what the firmware or test script was doing right before the reset. If possible, I would store the last state, error code, or counter in retained memory or non-volatile storage.
5. **Run long-duration stress tests.** Since the reset happens only once every few hours, I would run repeatable tests under different conditions: idle, high traffic, high power load, temperature variation, and interface-heavy activity.
6. **Isolate hardware interfaces one by one.** I would disconnect or disable peripherals and test UART, SPI, I2C, GPIO, PCIe, or other interfaces separately to check whether one bus or peripheral causes the reset.
7. **Use lab tools for evidence.** I would use a scope, logic analyzer, DMM, power supply, and simulation/test logs to correlate the reset with power behavior, interface activity, or firmware state.

I would change strategy when measurements stop supporting the first theory. For example, if the rails and reset line look clean, I would move toward firmware state-machine issues, watchdog servicing, memory corruption, interrupt timing, or rare interface corner cases. If the reset correlates with a specific peripheral or transaction, I would focus on signal integrity, protocol timing, clock-domain crossing, backpressure, or error handling.

A related lesson from my verification work is that bugs often appear in the boundary conditions. In my DMA and RTL verification work, issues like timing, arbitration, back-pressure, and corner-case protocol behavior required structured debugging instead of random trial-and-error.

4. What firmware/embedded engineers often do wrong

I think firmware and embedded engineers sometimes trust the happy path too much. A system may work in a short demo, but real hardware has unstable power, slow peripherals, noisy communication, timing variation, reset sequencing problems, and rare corner cases. Good embedded work should assume that things can fail and should include timeouts, retries, error codes, logging, and recovery paths.

One common saying I partly disagree with is: **“Keep the code as simple as possible.”** Simplicity matters, but code that is too simple without observability becomes hard to debug. I prefer code that is still readable, but includes good logging, clear state transitions, meaningful error handling, and enough instrumentation to understand what happened after a failure.

Another mistake is treating firmware as only software. Firmware is connected to power, timing, reset behavior, buses, sensors, registers, and physical hardware. The best firmware engineers are not just coders; they are system debuggers who understand both the software and the board.

5. A sketchy firmware or hardware “fix” that kept the project alive

One practical workaround I have used in hardware/software-style debugging is adding retries, timeout handling, and better logging around a communication or initialization path instead of assuming the first transaction would always succeed. It was not the cleanest solution because the deeper issue

could have been timing, sequencing, or an unreliable interface response, but it kept the test flow running and made the failure measurable.

The part I was proud of was that the workaround did not just hide the issue. It helped expose the issue more clearly by showing when the retry happened, how often it happened, and whether the system recovered or failed again. In early bring-up and validation, I think this type of controlled workaround can be useful as long as the team does not confuse it with the final root-cause fix.

A temporary fix should never replace proper debugging, especially for production firmware. But during validation, a small practical workaround can keep progress moving while collecting the evidence needed for the real fix.